

Database Diagram

Overdrive

Aventix
Epitech Paris
February 9, 2026

Summary

OverDrive – Database Diagram

0- Objectives

1- Global architecture: persistence vs real time

- 1- Persistent data (PostgreSQL)
- 2- Live data (excluding SQL)
- 3- Schema: Global architecture (PostgreSQL vs Redis vs Time-series)

2- PostgreSQL schema: main tables

- 1- **users**
- 2- **auth_sessions** (ou **refresh_tokens**)
- 3- **providers**
- 4- **provider_connections**
- 5- **events**
- 6- **sessions**
- 7- **drivers**
- 8- **user_presets**
- 9- **preset_widgets**
- 10- **race_highlights**
- 11- Schema: Global architecture (PostgreSQL vs Redis vs Time-series)

3- Real-time storage (excluding PostgreSQL)

- 1- Redis (recommended)
- 2- Time-series base (optional)
- 3- Diagram: Real-time structure Redis (keys)

4- Conclusion

Objective

The purpose of the OverDrive database is to store all the persistent information necessary for the operation of the product, while maintaining a **scalable** architecture compatible with **real-time** uses.

The database must cover the following needs :

- User and authentication session management
- Custom preset and layout management
- Provider (championship) and configuration management
- Race and session management (metadata, calendar, history)
- (Optional) Storage of key events, highlights, and replays

OverDrive is based on an important logic: high-frequency "live" data (telemetry, real-time state, positions updated very often) are not adapted to a classic SQL storage with each update. They are therefore processed via a separate system, mainly :

- **Redis** for the cache and the live state,
- An optional time-series base for history (InfluxDB or Timescale).

This separation guarantees an excellent compromise between performance, cost, and scalability.

1- Global architecture: persistence vs real time

The OverDrive database schema is based on a clear separation between:

1- Persistent data (PostgreSQL)

PostgreSQL is used to store all structured and durable data, including :

- User accounts,
- profiles,
- presets,
- course catalogue,
- provider configuration,
- key events.

This data is essential to the product, because it allows :

- Continuity of experience,
- personalization,
- multi-championship management,
- history and calendar navigation.

2- Live data (excluding SQL)

Live data is characterized by :

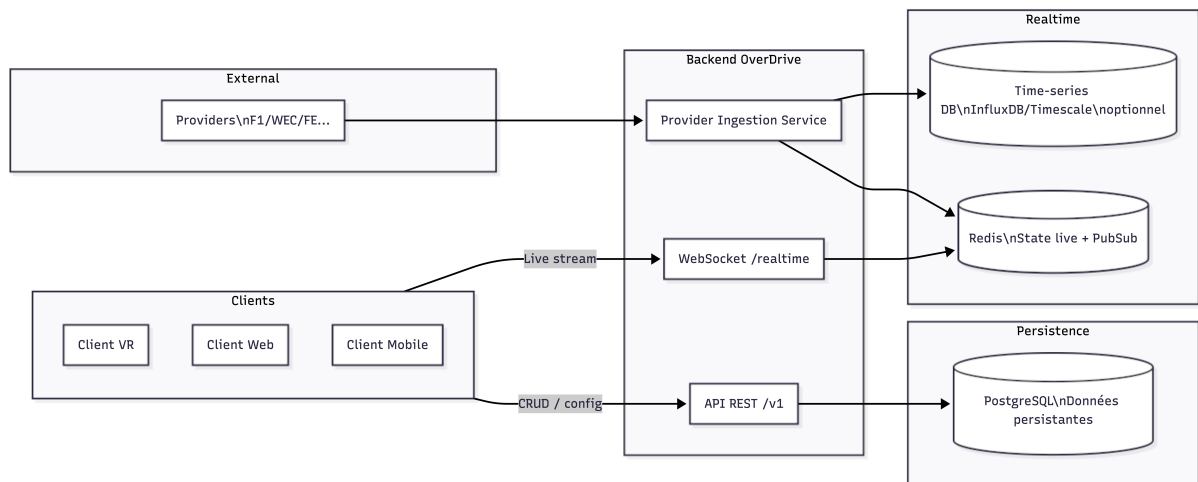
- A very high update frequency,
- Extremely low latency requirements,
- A large volume.

Storing this data directly in PostgreSQL with each update would create :

- significant costs,
- performance problems,
- unnecessary maintenance complexity.

For this reason, the live state is stored in **Redis**, and the telemetry history is possibly stored in a time-series database.

3- Schema: Global architecture (PostgreSQL vs Redis vs Time-series)



2- Main tables (PostgreSQL)

The SQL schema is designed to be **modular** and **scalable**, to easily add new providers, features, or widgets.

1- Table users

The **users** table is the heart of user management.

Elle stocke :

- l'identifiant unique de l'utilisateur,
- les informations de connexion,
- les informations de profil,
- les timestamps de création et modification.

Main fields :

- **id** (UUID, PK)
- **email** (unique)
- **display_name**
- **password_hash** (If classic authentication, optional if OAuth)
- **created_at, updated_at**

2- Table auth_sessions (or refresh_tokens)

OverDrive requires secure session management. For this, a dedicated table allows you to manage the persistence of connection via refresh tokens or long sessions.

Main fields :

- **id** (UUID, PK)
- **user_id** (FK → users.id)
- **refresh_token_hash**
- **expires_at**
- **created_at**

This separation between user data and session data allows :

- to secure token management,
- to limit the impact in case of a leak,
- to easily manage expirations,
- to revoke sessions without modifying the user profile.

3- Table providers

Since OverDrive is designed as a multi-championship platform, the **providers** table lists the available data sources. Each provider corresponds to a championship or data system: F1, WEC, FE, WRC, etc.

Main fields :

- **id** (UUID, PK)
- **code** (ex: f1, wec, fe) unique
- **name**
- **status** (active, beta, disabled)
- **created_at**

This model allows you to add a new championship without changing the overall structure of the base.

4- Table provider_connections

The **provider_connections** table represents the configuration of a provider for a user.

It allows you to store preferences by user, for example :

- regional settings,
- specific display options,
- feed preferences,
- data retrieval configuration.

Main fields :

- **id** (UUID, PK)
- **user_id** (FK)
- **provider_id** (FK)
- **config_json** (JSONB)
- **created_at, updated_at**

An important point is the management of sensitive data. As far as possible, OverDrive avoids storing external tokens in the database. If it becomes necessary, they must be encrypted, and stored separately in a secure system.

5- Table events

The **events** table corresponds to the catalog of races or events (e.g. Grand Prix, WEC round, rally, etc.).

It allows you to display:

- A calendar,
- Context information,
- The history,
- The race in progress or to come.

Main fields :

- **id** (UUID, PK)
- **provider_id** (FK)
- **season_year**
- **name** (ex: "Bahrain GP")
- **location**
- **start_time_utc**
- **end_time_utc** (nullable)
- **status** (scheduled, live, finished)
- **created_at**

6- Table sessions

An **event** can be composed of several sessions :

- Practice
- Qualifying
- Race

This separation is essential, because live timing and real-time data must be linked to a specific session.

Main fields :

- **id** (UUID, PK)
- **event_id** (FK)
- **type** (race, quali, practice)
- **status** (live, finished)
- **started_at_utc**, **ended_at_utc**

7- Table drivers

The **drivers** table constitutes a repository of drivers, by provider.

It is necessary because :

- The same driver can have a different ID depending on the provider;
- the application must display consistent information;
- it is important not to rely solely on live feeds for display.

Main fields :

- **id** (UUID, PK)
- **provider_id** (FK)
- **display_name**
- **team_name**
- **number** (nullable)

A possible extension of the scheme would be to add a **teams** table, in order to structure the teams more cleanly, but this is not essential in V1.

8- Table user_presets

The **user_presets** table stores each user's interface presets. A preset corresponds to an informational cockpit configuration: choice of widgets, layout, organization, etc..

Main fields :

- **id** (UUID, PK)
- **user_id** (FK)
- **name**
- **is_default** (bool)
- **created_at, updated_at**

This table allows the user :

- to create multiple configurations,
- to choose a default preset,
- to save settings between sessions.

9- Table preset_widgets

The **preset_widgets** table represents the composition of a preset. Each row corresponds to a widget displayed in the interface, with:

- its type,
- its configuration,
- its position and transformation in VR,
- its display order.

Main fields :

- **id** (UUID, PK)
- **preset_id** (FK)
- **widget_type** (ex : standings, telemetry, track, highlights)
- **settings_json** (JSONB)
- **pos_x, pos_y, pos_z** (float)
- **rot_x, rot_y, rot_z** (float)
- **scale** (float)
- **order_index** (int)

The combined use of dedicated fields (transform VR) and JSONB (specific parameters) allows great flexibility :

- Adding new widgets without changing the base,
- Storage of variable parameters according to the type of widget,
- Rapid adaptation to changes in the interface.

10- Table race_highlights

The **race_highlights** table allows you to store the significant events of a session:

- overtaking,
- pit stops,
- crash,
- flags,
- incidents.

Main fields :

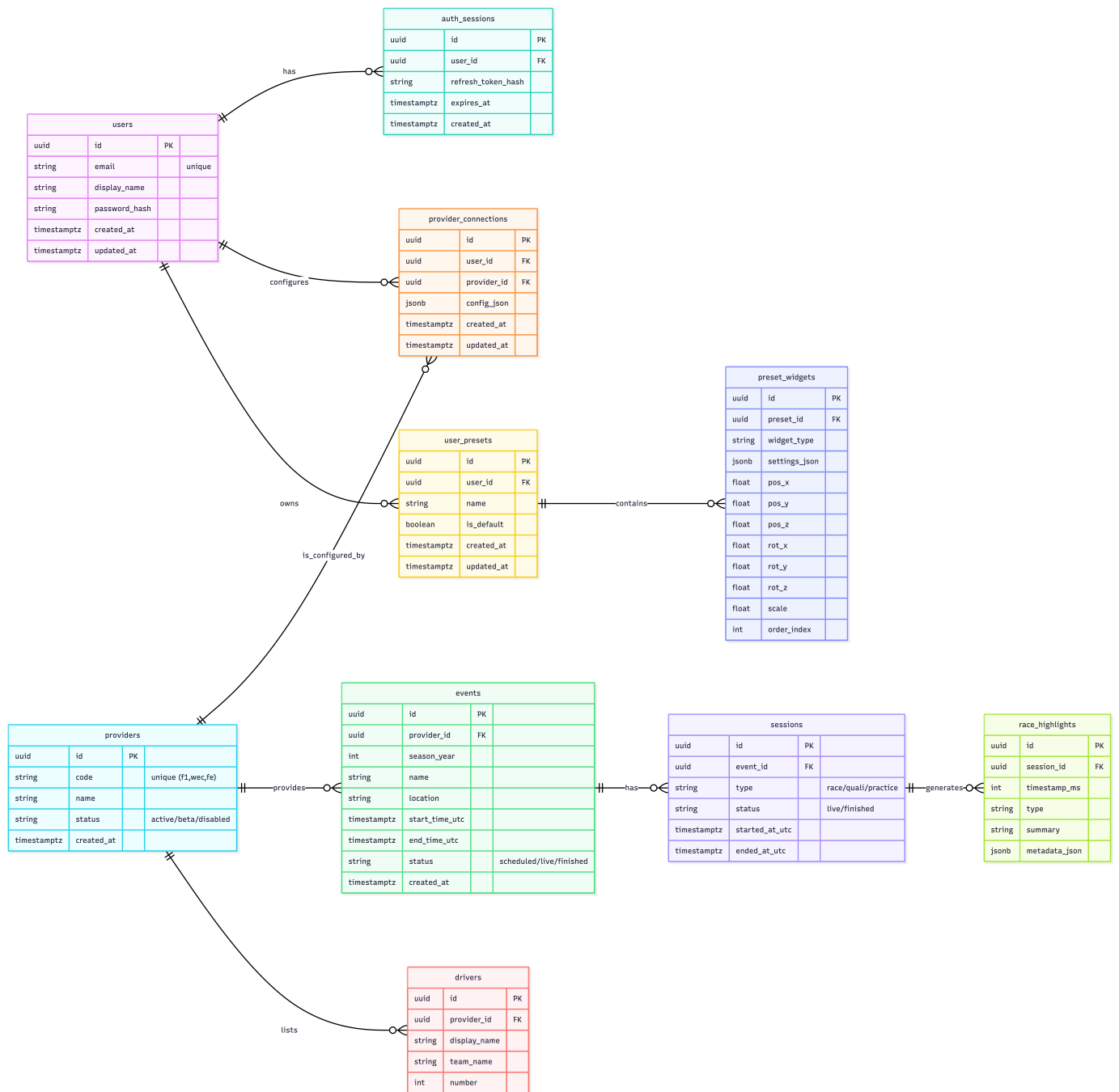
- **id** (UUID, PK)
- **session_id** (FK)
- **timestamp_ms**
- **type** (overtake, pit, crash, flag)
- **summary**
- **metadata_json** (JSONB)

This table is particularly useful for :

- to build a race “summary”,
- to allow quick navigation to key moments,
- to power a replay or analysis feature.

10- Shéma : ERD PostgreSQL (tables + relations)

Page 11



3. Real-time storage (excluding PostgreSQL)

As mentioned earlier, OverDrive distinguishes persistent storage from live storage.

1- Redis (recommended)

Redis is used to store and quickly distribute the live state.

Examples of keys :

- **realtime:session:<sessionId>:state**
(Ranking, flags, towers, overall status)
- **realtime:session:<sessionId>:driver:<driverId>**
(Last pilot state: position, time, tires, etc.)

Redis allows :

- Extremely low latency,
- Fast access for multiple concurrent customers,
- Effective real-time state management.

2- Time-series base (optional)

If OverDrive offers an advanced analysis or "replay data" functionality, a time series-oriented database can be added.

Possible solutions :

- InfluxDB
- Timescale (extension PostgreSQL)

This storage allows you to keep :

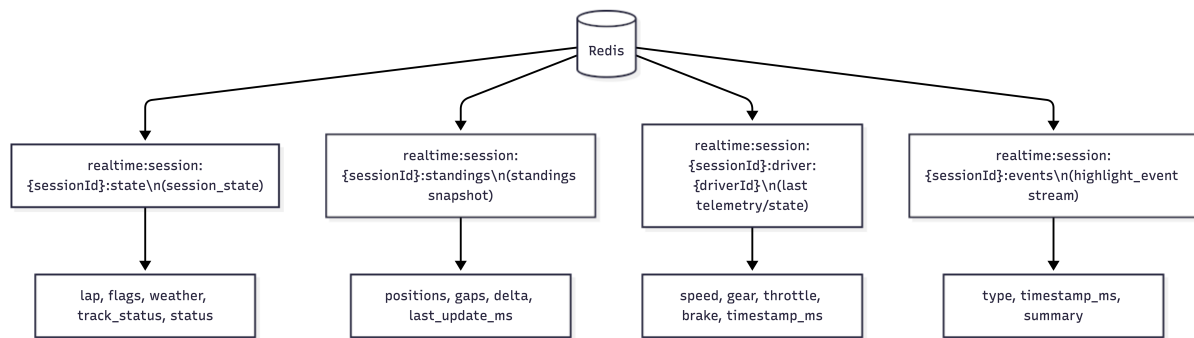
- Historical telemetry,
- performance curves,
- lap comparisons,
- post-race analyses.

This part is optional because it involves :

- More costs,
- more complexity,
- more data volume.

However, it is a natural extension for a data-oriented premium product.

3- Diagram: Real-time structure Redis (keys)



Conclusion

The OverDrive database schema is designed to be :

- **Modular:** easy addition of new providers and features,
- **Robust:** clear separation between persistence and live storage,
- **Scalable:** specialized storage according to use cases,
- **Product-oriented:** full support for customization via presets.

PostgreSQL covers all essential data (users, providers, events, presets), while Redis and an optional time-series database meet real-time constraints and future analysis needs.

This data architecture provides a solid foundation to support the evolution of OverDrive, its premium ambition, and its extension to several championships.